```
  has_one :profile
end
```

With that configuration you get a user's name via their profile, `user.profile.name`, but it could be handy to still be able to access such attribute directly:

```
class User < ActiveRecord::Base
  has_one :profile

  def name
    profile.name
  end
end
```

That is what `delegate` does for you:

```
class User < ActiveRecord::Base
  has_one :profile

  delegate :name, to: :profile
end
```

It is shorter, and the intention more obvious.

The method must be public in the target.

The `delegate` macro accepts several methods:

```
delegate :name, :age, :address, :twitter, to: :profile
```

When interpolated into a string, the `:to` option should become an expression that evaluates to the object the method is delegated to. Typically a string or symbol. Such an expression is evaluated in the context of the receiver:

```
# delegates to the Rails constant
delegate :logger, to: :Rails

# delegates to the receiver's class
delegate :table_name, to: :class
```

If the `:prefix` option is `true` this is less generic, see below.

By default, if the delegation raises `NoMethodError` and the target is `nil` the exception is propagated. You can ask that `nil` is returned instead with the `:allow_nil` option:

```
delegate :name, to: :profile, allow_nil: true
```

With `:allow_nil` the call `user.name` returns `nil` if the user has no profile.

The option `:prefix` adds a prefix to the name of the generated method. This may be handy for example to get a better name:

```
delegate :street, to: :address, prefix: true
```

The previous example generates `address_street` rather than `street`.

Since in this case the name of the generated method is composed of the target object and target method names, the `:to` option must be a method name.

A custom prefix may also be configured:

```
delegate :size, to: :attachment, prefix: :avatar
```

In the previous example the macro generates `avatar_size` rather than `size`.

Defined in `active_support/core_ext/module/delegation.rb`

3.8 Redefining Methods

There are cases where you need to define a method with `define_method`, but don't know whether a method with that name already exists. If it does, a warning is issued if they are enabled. No big deal, but not clean either.